

Chicago Quant Alley Crypto Trading Simulator Strategy Optimizer Assignment 2 Report

All python files and csv data sheets are available at: <https://github.com/Aarav-Malde/Chicago-Quant-Alley-Crypto-Trading-Simulator-Strategy-Optimizer/tree/main/Task%202%20-%20Quant%20Strategy%20Simulator>

What Each Module Does — Short Explanation

1. Simulator.py – The Core Simulation Engine

This file is the most important part. It manages everything from data loading, trade execution, and event simulation to PnL tracking.

`readData()`

Loads all CSVs from the data folder for each symbol and date in the configured range. Adds a “Symbol” column to each DataFrame and merges everything into one master DataFrame sorted by timestamp.

`startSimulation()`

Acts as the event engine. Iterates over each row (tick) of market data, updates the current price, and passes the row to the strategy via `onMarketData()`.

`onOrder()`

Simulates trade execution by applying slippage to the trade price. Updates current positions and buy/sell values accordingly. Notifies the strategy that the trade has been executed.

`computePnL() / printPnL()`

Calculates PnL using the formula:

$$\text{(Total Sell Value)} - \text{(Total Buy Value)} + (\text{Qty} \times \text{Current Price})$$

Used to track performance and print a final P&L report.

2. Strategy.py – The Trading Brain

This file contains the actual strategy logic and how it responds to market data and trade executions.

`__init__()`

Stores a reference to the simulator so it can place orders and access price/position information.

`onMarketData(row)`

Implements strategy logic. At 1 PM, it:

- * Reads the futures price
- * Selects the closest put and call options (within ~2%)
- * Sells 0.1 quantity of both to create a short straddle
- Then exits the position if:
 - * Futures price deviates >1% from entry
 - * Or total PnL exceeds +500 or drops below -500

`onTradeConfirmation()`

Updates running PnL and records trade history whenever a trade is executed by the simulator.

3. config.py – Global Settings

Contains user-defined parameters like:

- * Start and end date for the simulation
- * Symbols to include in the simulation
- * Path to the data folder

This modular config allows flexible switching between datasets and strategy setups.

4. stats/printStats.py – Post-Simulation Analytics

Analyzes the output PnL file (output_pnl.csv) and visualizes performance.

load_data()

Loads the CSV and checks for missing or malformed data.

compute_metrics()

Calculates key metrics:

- * Mean, median, standard deviation of PnL
- * Sharpe Ratio
- * Maximum Drawdown
- * Value at Risk (95%) and Expected Shortfall (95%)

plot_curves()

Plots:

- * Cumulative PnL over time
 - * Drawdown curve
- Saves both as PNGs and also displays them for easy inspection.

5. utils/getStrikes.py – (Optional Utility)

If implemented, this file can assist in selecting ATM (at-the-money) strikes closest to the current futures price. Useful when dynamically selecting strikes for option strategies.

6. `data/` – Raw Market Data

This folder contains:

- * One subfolder per trading day (e.g., `20240601`)
- * One `.csv` file per instrument (e.g., `MARK-BTCUSDT.csv`, `MARK-C-BTC-70000-20240601.csv`)

The simulator reads directly from this structure, so formatting must be strictly followed.

What Worked, What Didn't, and What Could Be Improved**

What Worked

1. Systematic Assignment Structure

The strategy simulator was structured in a very modular and well-scaffolded way. With clearly separated components for simulation (Simulator.py), strategy logic (Strategy.py), and

configuration (`config.py`), it allowed for clean experimentation and debugging. This separation of concerns encouraged a systematic approach to both coding and testing.

2. Interconnected `OnOrder` and `MarketData` Flow

The `onOrder()` and `onTradeConfirmation()` callbacks were tightly linked and logically consistent. This made tracking trade execution, slippage, and position updates fairly intuitive. Having the `Strategy` class receive a reference to the `Simulator` helped encapsulate behavior while still allowing necessary control over the simulation state.

3. Time-Based Entry-Exit Logic Worked Reliably

A simple yet effective example like the `**1 PM entry/exit logic**` demonstrated the power of a rule-based trading strategy. Implementing and backtesting such logic against time-sorted market data was straightforward and easy to test.

4. Efficient Data Handling with Pandas

Using Pandas for data loading, manipulation, and PnL tracking made the project significantly more efficient. Tasks like cumulative PnL calculation, drawdown computation, and time-series resampling were handled with just a few lines of vectorized code.

5. Performance Reporting with Visuals

Adding `printStats.py` at the end gave an analytical layer to the raw PnL outputs. The cumulative PnL and drawdown visualizations, along with metrics like Sharpe Ratio and VaR, added professional depth to the evaluation process.

What Didn't Work

1. Lack of Real Market Data

The project used placeholder or sample `.csv` files, not actual historical data from exchanges. As a result, the strategy couldn't be validated in a realistic market environment. This limited the credibility of both PnL curves and derived metrics like Sharpe ratio or drawdown.

2. Cross-Platform File System Issues

Windows OS does not support colons (:) in filenames, which caused initial read/write errors when dealing with time-formatted filenames (e.g., `"13:00"`). This needed platform-specific workarounds or reformatting of timestamp columns.

3. Simulation Granularity and Tick Frequency

The simulator operated on a row-by-row basis, treating each row as a market tick. However, without clear control over frequency (minute-wise, second-wise, etc.), the timing logic (like the 1 PM rule) was highly dependent on dataset formatting. This could cause inconsistent results on different datasets.

4. No Support for Transaction Costs or Commission Models

While slippage was included as a constant rate, real-world costs like commissions, funding rates, or spread-based execution weren't modeled. This makes the backtest slightly optimistic and not robust to microstructure noise.

5. PnL Calculation Assumes Immediate Execution

The `onOrder()` logic assumes trades are executed at the current market price with minor slippage. There is no handling of limit orders, partial fills, or latency, which are essential in real-world strategy simulation.

What I Would Improve

1. Add Support for Real Exchange Data

Use real historical crypto or equity data with proper OHLCV structure and timestamps. Integrating APIs like Binance, Alpha Vantage, or even backtest-ready datasets from Kaggle would bring realism.

2. Modularize Strategy Logic with Plug-in Design

Instead of hardcoding logic in `Strategy.py`, allow pluggable strategies through inheritance or JSON configs. This enables switching strategies (e.g., momentum, mean-reversion) without changing core code.

3. Support for Limit Orders and Order Book Depth

Enhance `onOrder()` to simulate limit/market orders, partial fills, and delays. Optionally simulate an order book to mimic real liquidity constraints.

4. Real-Time Plotting During Simulation

Include a live plot of cumulative PnL during the simulation, using tools like `matplotlib.animation` or `Plotly`. This adds interactivity and helps visualize how the strategy performs over time.

5. Enhanced Logging and Debug Mode

Add a logging module to track trade decisions, portfolio values, and signals at each tick. This would make debugging and validation much easier — especially during failure cases or unexpected PnL dips.

6. Unit Tests and Assertion Checks

Adding unit tests for PnL calculation, trade execution, and signal generation would improve robustness. Assertions for price/time alignment and valid order types would prevent silent errors.

7. Sharpe Ratio over Realistic Time Horizons

The Sharpe ratio was calculated using a naive frequency ($\sqrt{252}$). Aligning this with the actual data frequency (e.g., ticks per day or per hour) would yield more accurate risk-adjusted return assessments.